Spike: 3
Title: Goal-Oriented Action Planning (GOAP)
Author: Pattarapol Tantechasa, 103883220

Goals / deliverables:

- A functional **GOAP** simulation demonstrating intelligent agent control.
- An AI agent capable of dynamically formulating a multi-step plan to achieve a goal based on varying initial world states.
- Clear, observable evidence within the simulation of the agent weighing different action costs, time delays, or side effects to choose an optimal path.
- A well-documented **README.md** explaining how to run the code.

Technologies, Tools, and Resources used:

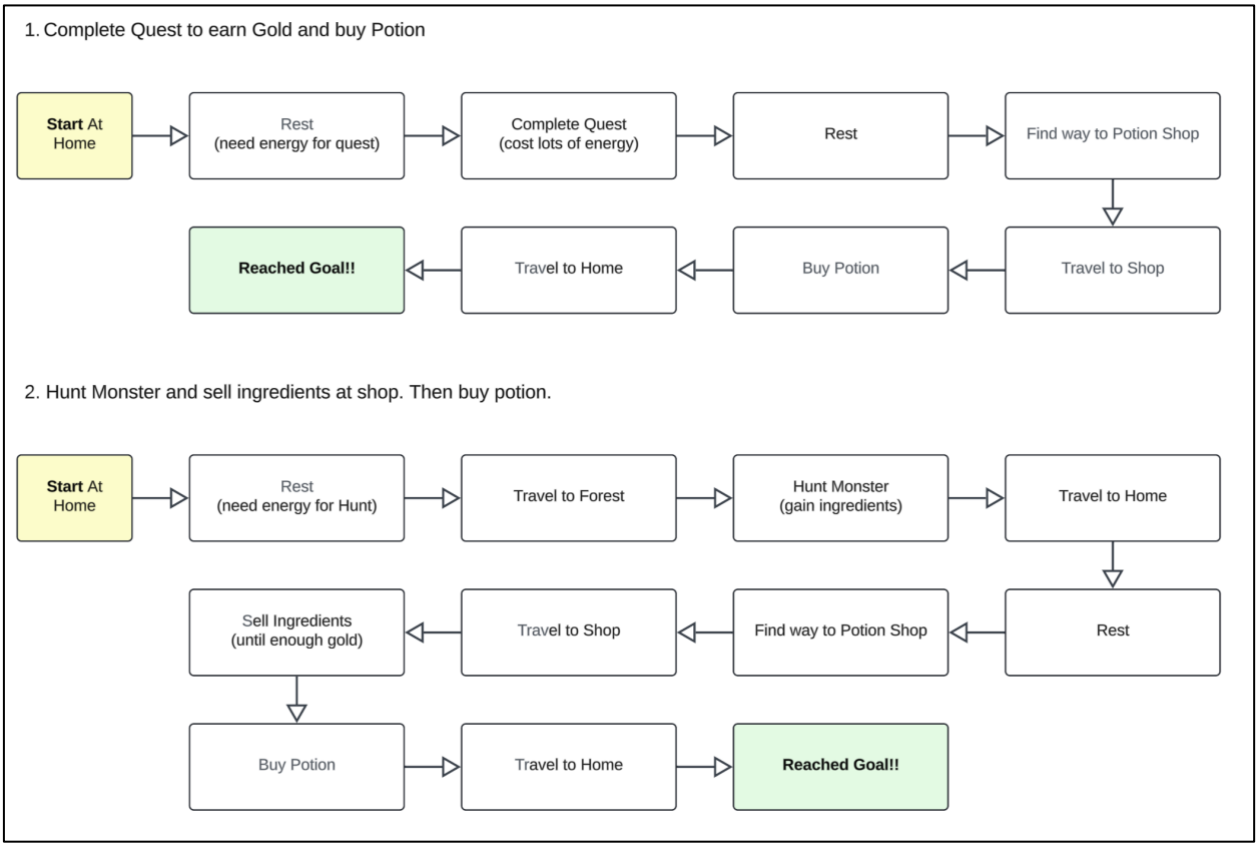
- Claude: brainstorm ideas, explain code and debugs.
- Visual Studio Code IDE
- Conda: Python environment manager

Tasks undertaken:

1. Download and Install **Visual Studio Code**
2. Install **Python** on your Machine
3. Open the main project downloaded
4. Open terminal and run cd command : **cd Assignments-Labs/Assignment-4/ Task-8-Spike-GOAP/wizard-journey**
5. Run: **python main.py**

Design Phase: Wizard Journey

Overview



A wizard needs to buy an MP potion before an adventure. They start with no money and limited energy. The system demonstrates GOAP by having the wizard plan a sequence of actions to reach the **goal: have 1 potion AND be home.**

Why This Works for GOAP

- Multiple valid paths (quest for money, or hunt and sell)
- Actions have dependencies (must find shop before traveling there)
- Resource management (energy, gold, ingredients)
- Preconditions force intelligent sequencing

Initial State

Variable	Value
Energy	50
Gold	0
Potions	0
Ingredients	0
Location	Home
Has Map	false
Quest Complete	false

Goal State

`potions >= 1 AND is_at_home = true`

The 11 Actions

#	Action	Cost	Effect
1	Rest	10	+50 energy (at home)
2	Travel to Shop	50	Go to shop (need map)
3	Travel to Home	50	Go home
4	Travel to Forest	50	Go to forest
5	Complete Quest	60	+30 gold (do once)
6	Hunt Monster	30	+20 ingredients
7	Sell Ingredients	5	+2 gold per ingredient
8	Find Potion Shop	20	Learn shop location
9	Buy Potion	10	Get potion (need gold > 10)
10	Gather Apple	10	+2 apples (in forest)
11	Eat Food	10	-10 hunger

File Structure

`main.py` — Entry point and scenario control
`world_state.py` — Stores world variables
`action.py` — Action class definition
`actions.py` — All 11 actions
`planner.py` — A* search algorithm
`simulation.py` — Step-by-step execution display

Outcomes and Reflection on GOAP vs SGI

Results

Scenario 1 (Quest Available): 7 steps, Cost 210

Rest → Complete Quest → Find Shop → Travel to Shop → Buy Potion → Travel Home

Scenario 2 (Quest Unavailable): 16 steps, Cost 320

Rest → Travel to Forest → Hunt → Travel Home → Find Shop → Travel to Shop → Sell x6 → Buy Potion → Travel Home

Both reach the same goal through different strategies.

How GOAP Works

1. Goal-Oriented: Planner works backward from goal (potions >= 1 AND home) to find required actions
2. Multi-Step Planning: Creates full sequences (7 or 16 steps) rather than reacting one step at a time
3. Cost Optimization: Chooses cheaper paths (Quest 210 vs Hunt 320)
4. Precondition Aware: Automatically inserts prerequisite actions (e.g., "Find Shop" before "Travel to Shop")
5. Adaptive: Same agent produces different plans based on world state (quest available vs unavailable)

GOAP vs SGI Comparison

Aspect	SGI (Reactive)	GOAP (Planning)
Strategy	Single-step reactions	Multi-step planning
Lookahead	None	Plans 7-16 steps ahead
Optimization	No	Cost-optimal via A*
Dependencies	Ignored	Respects preconditions
Adaptability	Hard-coded	Automatic replanning
Efficiency	Low	High

Key Difference

SGI would try to "go to shop" immediately without gold and fail. GOAP plans backward from goal and inserts prerequisite actions in the right order.

Conclusion

The wizard journey demonstrates GOAP advantages:

It automatically finds cost-optimal plans, adapts to world changes, respects action dependencies, and produces intelligent behaviour without hard-coded rules. The same agent creates different strategies (quest vs hunt) based on availability, proving planning is superior to reaction-based AI.